10PEARLS SHINE INTERNSHIP · DATA SCIENCES TRACK

Pearls AQI Predictor

A serverless system forecasting Karachi's Air Quality Index one, two, and three days out — the engineering decisions, dead ends, and one production bug that shaped it.

Location	Karachi, Pakistan · 24.8607°N, 67.0011°E
Repository	github.com/Saadjunaid0317/aqi-predictor
Live dashboard	aqi-predictor-nszy3iphgpcpwzmfejrxv2.streamlit.app
Training window	Aug 2023 – present · 26,000+ hourly rows
Status	Pipeline, registry, dashboard, explainability — live
Deadline	September 4, 2026

Compiled for record purposes · August 31, 2026

Contents

— Overview

01 Data & feature engineering

02 Automation

03 Model training & evaluation

04 Model registry

05 Live serving & the materialization bug

06 Dashboard

07 Explainability — "why this forecast?"

08 Known limitations

09 Key decisions

10 What's left

A Appendix — The build journey

Overview

Every hour, a GitHub Actions job pulls live weather and pollutant readings for Karachi from OpenWeather, computes the real US EPA Air Quality Index from the PM2.5/PM10 concentrations, and writes the result into a Hopsworks feature store. Three years of the same data was backfilled once from Open-Meteo to build a training set of roughly 26,000 hourly rows. Once a day, a second automated job retrains three independent Ridge regression models — one per forecast horizon — and registers them in Hopsworks' model registry. A Streamlit dashboard loads the latest models and shows the current reading, a 3-day forecast, and a per-prediction SHAP explanation of what's driving each number.

Forecast horizons	Best model	Automation
24h / 48h / 72h — day-average AQI, not a single point	Ridge Regression — beat RF, GB, XGBoost at every horizon	2 cron jobs — hourly features · daily training

01. Data & feature engineering

Two data sources feed the same feature schema. **OpenWeather** supplies the live hourly reading (temperature, humidity, wind speed, and PM2.5/PM10/CO/NO2/SO2/O3 concentrations) that the automation runs on. **Open-Meteo** supplied a one-time historical backfill covering roughly three years, used only to build a large enough training set — live automation never touches it again.

AQI itself isn't taken from either API as a pre-computed estimate. It's calculated directly from the EPA's piecewise-linear breakpoint formula applied to PM2.5 and PM10, taking whichever pollutant produces the higher sub-index as the AQI and the "dominant pollutant." Every record also carries time features (hour, day, month, day of week) and trend features: AQI 24/48/72 hours ago, the AQI's rate of change over the last 24 hours, and a rolling 24-hour mean and standard deviation.

> **Why compute AQI ourselves.** Neither API returns a directly comparable EPA AQI number, and using two different scoring conventions between the live feed and the historical backfill would have quietly poisoned the training set with an inconsistent target. Computing it once, in one function, for both paths keeps the label honest.

02. Automation

Two GitHub Actions workflows run the whole system without a server. Both commit their own output back into the repository, which turned out to matter more than it sounds — see §05.

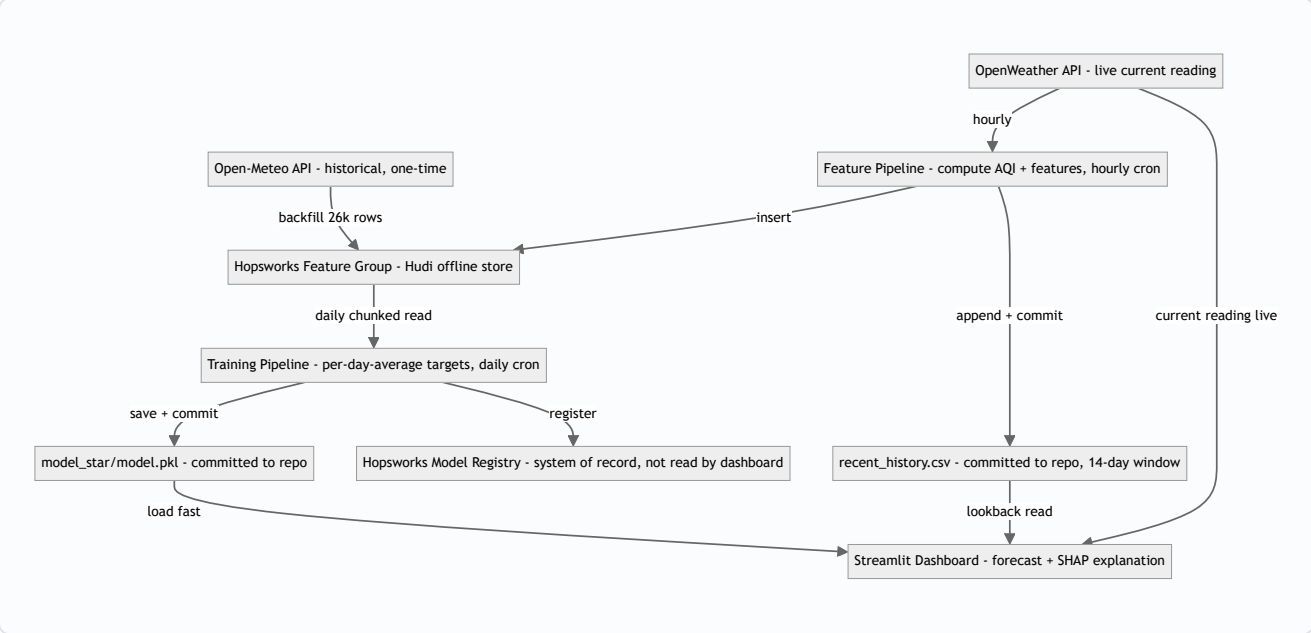

Fig. 1 — The full automated pipeline. The Model Registry is a terminal node: it's the official record required by the assignment, but the dashboard loads the committed `.pkl` files directly rather than calling Hopsworks on every page load.

The two scheduled workflows

Workflow	Schedule	Runs	Commits back
Hourly Feature Pipeline	`0 * * * *`	`push_to_hopsworks.py`	`recent_history.csv`
Daily Model Training	`0 3 * * *`	`register_models.py`	`model_*/model.pkl`, `shap_background.csv`

03. Model training & evaluation

The original design predicted a single AQI value at exactly 72 hours out. Following mentor and cohort discussion, the target was restructured into **three separate models**, each predicting the *average* AQI over one day's window — Day 1 (hours 1–24), Day 2 (25–48), Day 3 (49–72) — which is both closer to how AQI forecasts are conventionally reported and a simpler, more learnable target than one instantaneous far-future point. Each model is trained independently on the same feature set (a "direct" strategy), rather than feeding one model's output into the next, after a classmate found that recursive chaining compounds error badly by Day 3.

The real evaluation bar

Ridge, Random Forest, Gradient Boosting, and XGBoost were all trained and compared. The tree-based models went **negative R^2** at the 72-hour horizon — overfitting on a noisier long-range target — while Ridge stayed positive and simplest at every horizon. That pattern (Day 1 much stronger than Day 2/3, the simplest model winning) matched what nearly the whole cohort reported independently.

The mentor-confirmed registration criterion is **beating the persistence baseline** — "assume tomorrow looks like today" — not an absolute R^2 threshold. R^2 is mechanically capped low whenever the test window happens to be a calm, low-variance stretch of AQI, regardless of how good the model actually is, so persistence is the fairer bar.

Final results — Ridge regression, chronological 80/20 split (20,937 train / 5,235 test rows)

Horizon	RMSE	MAE	R^2	Persistence RMSE	Persistence R^2	Verdict
24h	**11.49**	8.26	0.569	16.76	0.083	**beats persistence**
48h	**14.48**	10.78	0.285	21.44	−0.567	**beats persistence**
72h	**15.02**	11.18	0.176	23.12	−0.952	**beats persistence**

A note on benchmarking against peers: a fellow intern on the same assignment reported a Random Forest reaching $R^2 = 0.997$ with RMSE ≈ 1.12. That combination is inconsistent with the natural noise level of a 3-day-ahead AQI forecast and reads as a strong signal of data leakage (most likely a lag feature computed too close to its own target, or a non-chronological split) rather than a genuinely better model — the same conclusion reached earlier about a different classmate's suspiciously strong SVR/CatBoost numbers. Neither was worth chasing.

04. Model registry

Each of the three trained models is registered in the Hopsworks Model Registry as `aqi_ridge_24h`, `aqi_ridge_48h`, and `aqi_ridge_72h`, tagged with its RMSE, MAE, R^2, and the persistence RMSE it beat, alongside an input-example schema. The daily training workflow (§02) registers a new version every run, so the registry accumulates a version history rather than overwriting in place — currently at version 2 for all three models after the first scheduled retrain.

05. Live serving & the materialization bug

This section is the one real production bug this project hit, and the fix ended up being more instructive than the model comparison.

Despite hundreds of successful hourly inserts, querying Hopsworks for anything from roughly the last one to two weeks reliably returned **zero rows** — confirmed with multiple query methods, including the same chunked-read function that works perfectly for the three-year historical set. The pipeline logs eventually surfaced the actual mechanism directly:

> **Observed in production, every hourly run:**
> `UserWarning: Materialization job is already running, aborting new execution. Please wait for the current execution to finish before triggering a new one.`

Hopsworks' Hudi-backed offline store needs a background "materialization" job to make newly inserted rows efficiently queryable. On the free tier, that job appears to be perpetually behind the hourly insert cadence — each new run tries to trigger it and is told one is already in progress. It isn't a transient blip; it's a standing backlog.

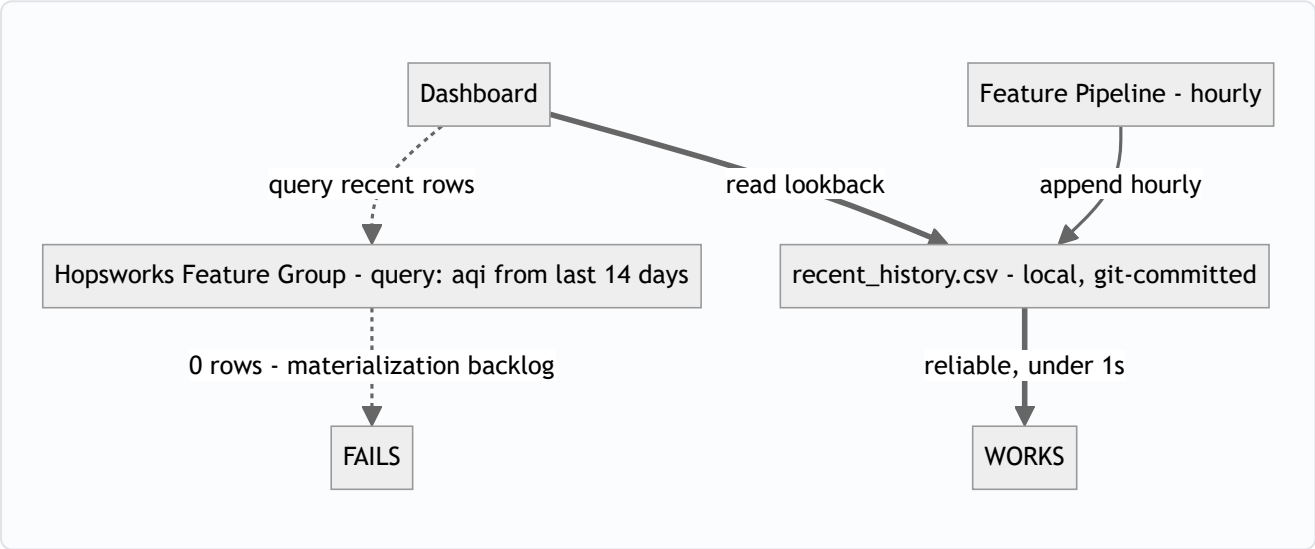

Fig. 2 — The two paths the dashboard could take for recent AQI history. The dotted path is what the architecture nominally calls for; the solid path is what actually works, fed by the same hourly job that already computes each record before sending it to Hopsworks.

The fix doesn't touch the required architecture: Hopsworks stays the feature store of record, every hourly run still inserts into it, and training still reads from it exclusively (older data has fully materialized by the time it's needed). The hourly script already has each record in memory the moment it computes it, so it now also appends that record to a small local CSV and commits it back to the repo — a live-serving cache that happens to live in git, purely to work around a platform-side query limitation for data younger than about two weeks.

06. Dashboard

> **A note on how this was built.** Web/frontend development is not my background — my focus for this internship is data science, not UI engineering. I used AI assistance (Claude) heavily for the Streamlit dashboard itself: the layout, CSS/styling, animations, and Plotly chart configuration. The data pipeline, feature engineering, model training/evaluation, and the SHAP explainability logic are my own work and understanding; the dashboard's visual presentation is where I leaned on AI help the most, since it falls outside my domain.

A Streamlit app shows the current reading on a color-coded EPA gauge, six live weather/pollutant stat tiles, and three forecast cards (one per horizon) with a category badge and a delta arrow against the current reading. A combined chart plots recent actual AQI against the 3-day forecast on one timeline, with a dashed line marking the transition from history to prediction. AQI severity colors are always paired with a text label and an icon — never color alone — since the standard EPA palette fails contrast and colorblind-safety checks when used as a solid fill.

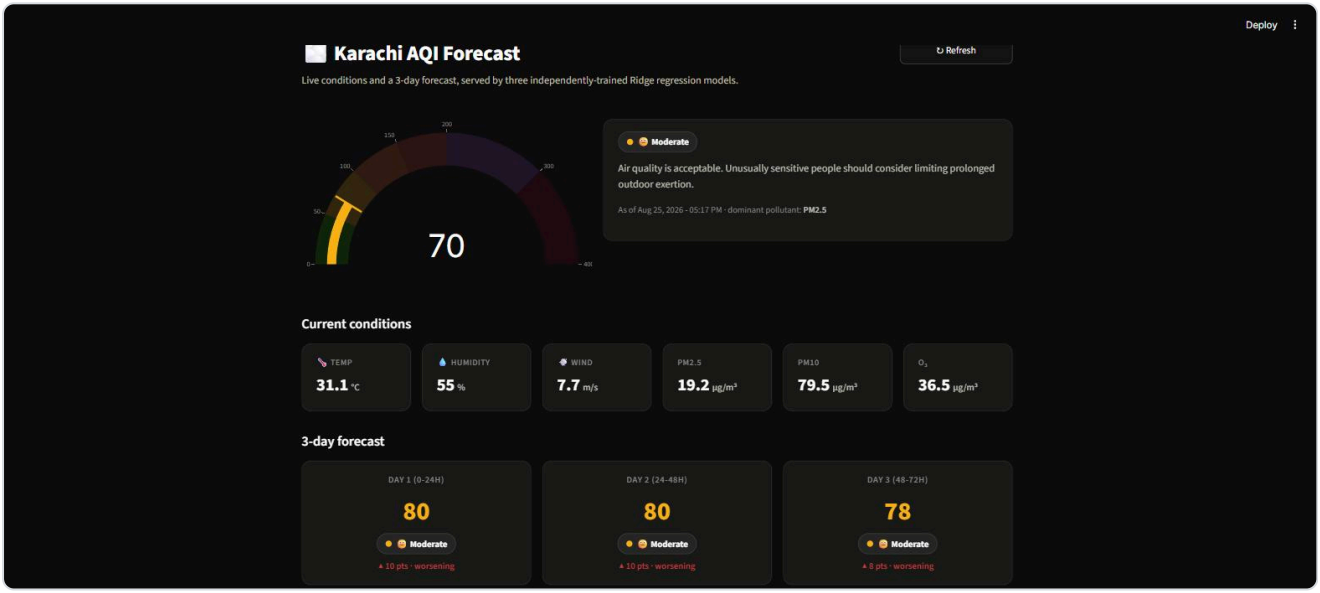

Fig. 3 — The dashboard's live view, captured during development. Models load from the committed `.pkl` files (fast, no Hopsworks round-trip); the current reading comes from OpenWeather live; the lookback history comes from `recent_history.csv`.

07. Explainability — "why this forecast?"

Each forecast card links to a SHAP breakdown showing exactly which features pushed that day's number up or down. Because all three models are linear (Ridge), the explanation is exact rather than approximated: `shap.LinearExplainer` computes each feature's contribution as its coefficient times its deviation from a background average, so the baseline plus the sum of all contributions always equals the model's actual prediction — verified directly (base value + ΣSHAP = prediction, to six decimal places) before shipping the panel.

The background distribution is a 150-row sample spread evenly across the full ~3-year training period (not a random sample, which could over-represent any one season), refreshed automatically every time the daily training job pulls fresh data.

> **Why this, and not a bigger model.** A peer's project on the same assignment used SHAP for per-prediction explanations, which is a genuinely strong addition to a forecasting dashboard — their headline model accuracy claim wasn't credible (see §03), but the explainability idea was worth adopting on its own merits.

08. Known limitations

- **Cold-start lookback** — The `recent_history.csv` cache only starts accumulating from the moment it was introduced. 24-hour lookback is accurate within a day of that point; full 72-hour fidelity needs three days to build up. Until then the dashboard reports how far off its nearest available match is, rather than hiding the gap.
- **R^2 in calm periods** — Low R^2 on a low-variance test window is an artifact of that window's AQI barely moving, not evidence of a weak model — which is exactly why persistence, not an absolute R^2 target, is the registration bar (§03).
- **SHAP background is a sample** — 150 evenly-spaced rows approximate the training distribution well but aren't the full ~26,000-row set; contribution magnitudes are stable but not bit-identical to a full-background explainer.
- **Daily retraining has a 72-hour lag** — A row only becomes usable training data once its full 72-hour-ahead target window exists, so the daily job won't see genuinely new usable rows until data ages past that point — confirmed directly when the first scheduled retrain produced byte-identical models to the previous run.
- **Materialization backlog is unresolved, not fixed** — The workaround in §05 makes live serving reliable; it doesn't make Hopsworks' offline store queryable for recent data. Anything that genuinely needs a Hopsworks-side read of the last two weeks (not just the dashboard's own lookback) would still hit the same wall.

09. Key decisions

Decision	Reasoning
Per-day-average, 3-separate-model targets	Matches mentor- and cohort-confirmed correct structure; simpler and more learnable than one instantaneous 72h point.
Direct strategy, not recursive chaining	A classmate found chaining compounds error and makes Day 2/3 measurably worse.
Stopped tuning once Ridge beat persistence everywhere	Matches the real registration criterion; protects time for the rest of the pipeline.
Skepticism toward suspiciously strong peer results	RMSE/R^2 combinations mathematically inconsistent with realistic AQI noise — a leakage signal, not a technique to copy.
Local `recent_history.csv` cache for live serving	Hopsworks stays the true feature store; this only works around a confirmed materialization-lag query limitation, purely for the live app.
SHAP via the closed-form linear explainer	Exact for a linear model, no extra retraining, and the identity base + ΣSHAP = prediction is independently checkable.

10. What's left

Against the original brief's final-submission checklist:

- ✓ End-to-end prediction system
- ✓ Automated, scalable pipeline
- ✓ Interactive dashboard — publicly deployed
- ✓ This report
- ✓ Hazardous-AQI alerting (dashboard banner + automated alert log)
- ✓ Deep learning model (LSTM) evaluated alongside the statistical/ensemble models
- ✓ A full project write-up covering the API selection, data cleaning, feature engineering, every model tried and its results, the reasoning behind the final model, and every blocker hit along the way — see Appendix, and the repo's `EDA_Writeup.md`

An LSTM (24-hour input window, scaled features, small 32-unit layer to avoid overfitting the ~26k-row dataset) was added to `train_model.py`'s comparison, evaluated with the same RMSE/MAE/R^2 as Ridge/Random Forest/Gradient Boosting/XGBoost on the same held-out chronological test split. It won on the 24h and 72h horizons (R^2 0.592 vs Ridge's 0.569, and 0.208 vs 0.176) and came close on 48h (0.280 vs Ridge's 0.285). Production still registers Ridge (`register_models.py` is unchanged) — the margins are small enough, and Ridge's simplicity/interpretability/fast retraining make it the better fit for a serverless daily-retrain setup, but the comparison satisfies the brief's "statistical to deep learning" guideline and is documented here rather than assumed.

Appendix — The build journey

The sections above are the finished-system report. This appendix pulls a few highlights from the full build journey — written in the moment, in plain first-person language — to show more of the actual process, not just the result. The complete version, including every blocker hit along the way, lives in `EDA_Writeup.md` in the repository.

Picking the API

I went with **OpenWeather** for two reasons: one API key covers both the weather data (temperature, humidity, wind speed) and the pollution data (PM2.5, PM10, CO, NO2, SO2, O3) I needed, and it has a proper historical endpoint. I used coordinates (24.8607°N, 67.0011°E) instead of a city name to avoid any ambiguity. When OpenWeather's historical access turned out to be limited on the free plan, I switched to **Open-Meteo** just for the one-time three-year backfill — both feed into the exact same feature schema, so it doesn't matter which one a row came from once it's processed.

One thing I made a deliberate call on: I didn't use the AQI number either API returns directly. OpenWeather's own AQI is a 1-to-5 scale, not the real 0-500 EPA scale. So I calculate AQI myself from the raw PM2.5/PM10 concentrations using the official EPA breakpoint formula, applied identically to both the live feed and the historical backfill — using two different scales between them would have quietly poisoned the training target.

All five models, side by side

Here is the full comparison across every model tried, not just the one that shipped:

Horizon	Model	RMSE	MAE	R^2
24h	Persistence (baseline)	16.76	—	0.083
24h	Ridge	11.49	8.26	0.569
24h	Random Forest	12.08	8.64	0.523
24h	Gradient Boosting	11.81	8.33	0.544
24h	XGBoost	11.98	8.41	0.532
24h	**LSTM**	**11.18**	**8.15**	**0.592**
48h	Persistence	21.44	—	−0.567
48h	Ridge	14.48	10.78	0.285
48h	Random Forest	15.96	11.73	0.132
48h	Gradient Boosting	15.74	11.55	0.156
48h	XGBoost	15.45	11.35	0.187
48h	LSTM	14.54	10.83	0.280
72h	Persistence	23.12	—	−0.952
72h	Ridge	15.02	11.18	0.176
72h	Random Forest	16.93	12.74	−0.047
72h	Gradient Boosting	17.00	12.70	−0.056
72h	XGBoost	16.99	12.60	−0.054

72h	**LSTM**	**14.73**	**11.08**	**0.208**

The tree-based models go negative R^2 at 72h — overfitting on the noisiest horizon. The LSTM actually wins on 24h and 72h, and comes close on 48h, but Ridge stays the production model: the win margin is small, and for a system that retrains itself daily with nobody watching, a small, fast, easy-to-explain model beats a slightly-more-accurate one that needs a heavier dependency, scaled/reshaped input, and longer retraining.

Blockers hit along the way, and how they got fixed

- **Windows + conda not talking to each other.** VS Code's terminal didn't recognize `conda` until running `conda init powershell` once from a separate Anaconda Prompt.
- **Windows-specific Hopsworks install headaches.** Build failures on a package called `twofish`, issues with `pyarrow`, and Hopsworks code assuming a Linux-style `/tmp` folder Windows doesn't have — worked through one at a time until the pipeline ran reliably.
- **Reading 26,000 rows from Hopsworks in one request was unreliable.** Fixed by splitting the read into 60-day time-windowed chunks instead of one large pull.
- **The materialization-lag bug** — the project's one real production bug, detailed in full in §05.
- **Two automated jobs racing to push to GitHub.** The hourly and daily workflows sometimes collided; fixed with a `git pull --rebase` before each push.
- **Streamlit Cloud deployment failures.** A removed Python module (`imp`) broke the old `hopsworks` dependency's install script. The fix was realizing the dashboard itself never imports `hopsworks` — only the backend automation scripts do — so it was removed from the shared requirements file and installed separately only where it's actually needed.
- **A transient "socket closed" training failure.** A one-off Hopsworks network hiccup killed a daily training run; fixed with a short retry-with-backoff around the chunked read.
- **Suspiciously perfect peer results.** A couple of classmates reported numbers (like $R^2 = 0.997$) that don't add up mathematically for a 3-day-ahead AQI forecast — a near-certain data-leakage signal, not a technique worth copying.